

MOMO 项目 Day 1 搭建指令

给 AI 助手（Claude Code / Cursor）的执行指令

这是 MOMO 项目（中文场景化 AI 情绪陪伴 App）第一天的搭建任务。目标是搭好 monorepo 骨架，每个部分都能"跑起来"但都是空壳。今天不写业务代码，明天再做。

项目背景（必读）

MOMO 是面向中文用户的场景化 AI 情绪陪伴 App，核心场景是深夜低落、情绪内耗、关系复盘、压力峰值、孤独陪伴。

不是治疗产品、不是角色扮演、不是泛聊天工具。

已确定的技术决策（不要质疑，不要改）

- 后端: Python 3.12 + FastAPI + Pydantic + Postgres + Redis
- 包管理: uv (Python) + pnpm (JS)
- 移动端: React Native (Expo) + TypeScript
- 抢先页: Next.js + Tailwind（一页介绍 + 邮箱收集，不是完整网站）
- LLM: DeepSeek（主）+ 豆包（备份），通过 provider 抽象层切换
- 部署目标: 阿里云 ECS + Docker
- 本地系统: macOS

工作原则

- 严格按顺序执行每个 Step
- 每一步执行完做验证，验证不通过不要进入下一步
- 遇到问题先停下报告给用户，不要自行猜测解决
- 不要添加额外功能、不要"优化"、不要修改既定的技术选型
- 所有文件路径以仓库根目录 `momo/` 为基准

Step 1：检查工具链

执行下列命令，确认每个工具都已安装并记录版本：

```
bash
```

```
python3.12 --version    # 应该是 3.12.x
uv --version
node --version          # 应该是 v20.x
pnpm --version
git --version
```

如果有缺失的工具，按下面安装：

```
bash

# Homebrew (如果没装)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Python 3.12
brew install python@3.12

# uv (Python 包管理器)
curl -LsSf https://astral.sh/uv/install.sh | sh
# 安装后需要重启 terminal 或 source ~/.zshrc

# Node.js 20 LTS
brew install node@20

# pnpm
brew install pnpm

# Expo CLI (全局)
pnpm add -g expo-cli eas-cli
```

验证：上面五个 `--version` 命令都有正常输出。

Step 2：创建 monorepo 骨架

2.1 在用户家目录创建项目（如果用户没有指定其他位置）

```
bash

cd ~
mkdir momo
cd momo
```

2.2 创建目录结构

```
bash

mkdir -p apps/mobile
mkdir -p apps/web
mkdir -p services/api
mkdir -p packages/shared-types
mkdir -p docs
mkdir -p .claude
```

2.3 创建顶层占位文件

```
bash

touch README.md
touch .cursorrules
touch .gitignore
touch CLAUDE.md
```

验证：`tree -L 2 -a -I '.git'` 应能看到完整的目录结构。如果没装 `tree`，用 `ls -la` 检查也行。

Step 3：初始化后端（FastAPI）

3.1 进入后端目录并初始化

```
bash

cd services/api
uv init --name momo-api --python 3.12
```

3.2 删除 uv 默认生成的样例文件

```
bash

rm -f main.py hello.py
```

3.3 添加生产依赖

```
bash
```

```
uv add fastapi 'uvicorn[standard]' pydantic pydantic-settings python-dotenv httpx sc
```

3.4 添加开发依赖

```
bash
```

```
uv add --dev pytest pytest-asyncio ruff mypy
```

3.5 创建后端目录结构

```
bash
```

```
mkdir -p app/core app/api app/domain app/llm app/infra
mkdir -p app/domain/conversation app/domain/memory app/domain/persona app/domain/saf
mkdir -p tests
```

```
# Python 包标记
```

```
touch app/__init__.py
touch app/core/__init__.py
touch app/api/__init__.py
touch app/domain/__init__.py
touch app/domain/conversation/__init__.py
touch app/domain/memory/__init__.py
touch app/domain/persona/__init__.py
touch app/domain/safety/__init__.py
touch app/domain/reflection/__init__.py
touch app/llm/__init__.py
touch app/infra/__init__.py
touch tests/__init__.py
```

3.6 创建主入口 `app/main.py`

```
python
```

```
"""FastAPI 应用入口"""
from contextlib import asynccontextmanager

import structlog
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.core.config import settings

logger = structlog.get_logger()

@asynccontextmanager
async def lifespan(app: FastAPI):
    """应用启动/关闭时的钩子"""
    logger.info("momo_api_starting", env=settings.env)
    yield
    logger.info("momo_api_stopping")

app = FastAPI(
    title="MOMO API",
    description="场景化 AI 情绪陪伴后端",
    version="0.1.0",
    lifespan=lifespan,
)

# CORS: 开发阶段允许所有, 上线前必须收紧
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
async def root():
    return {"service": "momo-api", "status": "ok"}

@app.get("/health")
```

```
async def health():  
    return {"status": "healthy", "env": settings.env}
```

3.7 创建配置文件 `app/core/config.py`

python

```
"""应用配置 - 通过环境变量加载"""
from typing import Literal

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """全局配置。所有敏感配置通过 .env 文件加载。"""

    # 环境
    env: Literal["dev", "staging", "prod"] = "dev"
    debug: bool = True

    # CORS
    cors_origins: list[str] = ["*"] # 生产环境必须收紧

    # 数据库（暂未启用）
    database_url: str = "postgresql+psycopg://momo:momo@localhost:5432/momo"

    # Redis（暂未启用）
    redis_url: str = "redis://localhost:6379/0"

    # LLM 供应商（暂未启用，明天接）
    deepseek_api_key: str = ""
    doubao_api_key: str = ""

    # 内容安全（暂未启用）
    aliyun_access_key_id: str = ""
    aliyun_access_key_secret: str = ""

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
    )

settings = Settings()
```

3.8 创建 `.env.example` 模板

```
bash
```

```
ENV=dev
DEBUG=true

# 数据库
DATABASE_URL=postgresql+psycopg://momo:momo@localhost:5432/momo

# Redis
REDIS_URL=redis://localhost:6379/0

# LLM
DEEPSEEK_API_KEY=
DOUBAO_API_KEY=

# 阿里云内容安全
ALIYUN_ACCESS_KEY_ID=
ALIYUN_ACCESS_KEY_SECRET=
```

文件路径：`services/api/.env.example`

3.9 复制一份本地用的 .env

```
bash

cp .env.example .env
```

3.10 创建后端的 `.gitignore`

文件路径：`services/api/.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/
.ruff_cache/
*.egg-info/
```

3.11 启动验证

```
bash

uv run uvicorn app.main:app --reload --port 8000
```

验证（在新 terminal 窗口）：

```
bash

curl http://localhost:8000/health
# 应返回: {"status":"healthy","env":"dev"}

curl http://localhost:8000/
# 应返回: {"service":"momo-api","status":"ok"}
```

也可以浏览器访问 <http://localhost:8000/docs> 看 FastAPI 自动生成的文档。

验证通过后 Ctrl+C 停止服务器，回到根目录：

```
bash

cd ../../
```

Step 4：初始化移动端（Expo）

4.1 进入移动端目录并创建 Expo 项目

```
bash

cd apps/mobile
pnpm create expo-app . --template blank-typescript
```

注意：执行时如果遇到 "directory is not empty" 提示，选 yes 继续。

4.2 添加核心依赖

```
bash

pnpm add @tanstack/react-query zustand expo-secure-store
```

4.3 启动验证

```
bash

pnpm start
```

验证：终端会显示二维码界面。

- 如果用户的 Mac 装了 Xcode：按 **(i)** 应能启动 iOS 模拟器并显示默认页面
- 如果用户没装 Xcode：让用户在手机装 Expo Go App 扫码（这一步可让用户自己做）

看到 "Open up App.tsx to start working on your app!" 文字就是成功。

验证通过后 Ctrl+C 退出，回到根目录：

```
bash
cd ../../
```

Step 5：初始化抢先体验页（Next.js）

5.1 进入网页目录并创建 Next.js 项目

```
bash
cd apps/web
pnpm create next-app@latest . --typescript --tailwind --app --no-src-dir --import-alias
```

注意：遇到 "directory is not empty" 选 yes。

5.2 启动验证

```
bash
pnpm dev
```

验证：浏览器打开 <http://localhost:3000>，应能看到 Next.js 默认欢迎页。

验证通过后 Ctrl+C 退出，回到根目录：

```
bash
cd ../../
```

Step 6：写好 `.cursorrules`

文件路径：项目根目录 `.cursorrules`

MOMO 项目 - Cursor 规则

项目本质

MOMO 是面向中文用户的场景化 AI 情绪陪伴 App。

核心场景：深夜低落、情绪内耗、关系复盘、压力峰值、孤独陪伴。

不是治疗产品，不是角色扮演产品，不是泛聊天产品。

仓库结构 (monorepo)

- apps/mobile: React Native (Expo) 移动 App
- apps/web: Next.js 抢先体验页 (一页介绍 + 邮箱收集)
- services/api: FastAPI 后端 (Python 3.12 + uv)
- packages/shared-types: 前后端共享类型

技术栈

- 后端: Python 3.12 + FastAPI + Pydantic + Postgres + Redis
- 前端: React Native (Expo) + TypeScript + Zustand + React Query
- LLM: DeepSeek (主) + 豆包 (备份), 通过 provider 抽象层切换
- 部署: 阿里云 ECS + Docker (一开始单机即可)

代码约定

- 后端按业务领域切分目录 (domain/conversation, domain/memory 等), 不要按技术类型
- 所有 LLM 调用必须经过 app/llm/ 的 provider 接口, 不要直接调 SDK
- 所有用户输入和模型输出必须经过 app/domain/safety/ 的内容审核
- Pydantic 模型用于所有 API schema 和 LLM 结构化输出
- 数据库迁移用 Alembic (明天加)
- 前端通过 React Query 管理服务端状态, 本地状态用 Zustand
- 移动端用 Expo OTA, 避免每次小改都过 App Store 审核

不要做的事

- 不要用同步 HTTP 客户端 (必须用 httpx async)
- 不要在业务代码里硬编码模型名 (写在 config 里)
- 不要把秘钥写进代码 (用 .env)
- 不要做未成年人主场景, 所有 UI 都默认成年人
- 不要用任何 "治疗" "诊断" 措辞, 统一用 "情绪支持/陪伴/自我整理"
- 不要在没有 type hint 的情况下写 Python 函数
- 不要在没有 TypeScript 类型的情况下写前端函数
- 不要直接 import 未抽象的 LLM SDK (用 app/llm/ 的接口)

验证命令

- 后端格式化: cd services/api && uv run ruff format .
- 后端类型检查: cd services/api && uv run mypy app
- 后端测试: cd services/api && uv run pytest
- 后端启动: cd services/api && uv run uvicorn app.main:app --reload

- 移动端启动: `cd apps/mobile && pnpm start`
- 网页启动: `cd apps/web && pnpm dev`

关键业务概念（写代码必须理解）

- 场景 (Scene): 深夜低落、情绪内耗、关系复盘、压力峰值、孤独陪伴 - 用户进入 App 时选择
- 人格卡 (Persona): MOMO 的稳定语气、边界、价值观、用户称呼
- 情景记忆 (Episodic Memory): 事件、情绪、触发点、回应偏好（不是简单对话历史）
- 回应强度 (Response Intensity): 轻回应 / 陪伴 / 复盘 / 建议 / 行动 - 不是越多越好
- 聊后整理 (Reflection Card): 情绪命名、触发点、下一步轻动作
- 安全降级 (Safety Fallback): 危机表达时从陪伴模式切到现实支持

Step 7：写好 README

文件路径：项目根目录 `README.md`

markdown

MOMO

中文场景化 AI 情绪陪伴 App。

项目结构

```
```\nmomo/
├─ apps/
│ ├─ mobile/ # React Native (Expo) - 主 App
│ └─ web/ # Next.js - 抢先体验页
├─ services/
│ └─ api/ # FastAPI 后端
├─ packages/
│ └─ shared-types/ # 前后端共享类型
└─ docs/ # 项目文档
```\`
```

本地开发

前置要求

- Python 3.12 (推荐用 `uv` 管理)
- Node.js 20+
- pnpm
- Docker Desktop (用于本地 Postgres + Redis, 明天用)

启动后端

```
```\`bash
cd services/api
cp .env.example .env # 第一次需要
uv sync # 安装依赖
uv run uvicorn app.main:app --reload
```\`
```

后端跑在 <http://localhost:8000>, 文档在 <http://localhost:8000/docs>。

启动移动端

```
```\`bash
cd apps/mobile
pnpm install
pnpm start
```\`
```

按 ``i`` 启动 iOS 模拟器，或在手机用 Expo Go 扫码。

启动抢先体验页

```
````bash
cd apps/web
pnpm install
pnpm dev
````
```

跑在 `http://localhost:3000`。

技术栈

- **后端**：Python 3.12 + FastAPI + Pydantic + Postgres + Redis
- **移动端**：React Native (Expo) + TypeScript
- **抢先页**：Next.js + Tailwind
- **LLM**：DeepSeek (主) + 豆包 (备份)
- **部署**：阿里云 ECS + Docker

业务核心概念

- **场景**：用户进入时选择当前状态（深夜低落 / 情绪内耗 / 关系复盘 / 压力峰值 / 孤独陪伴）
- **人格卡**：MOMO 的稳定语气、边界、价值观
- **情景记忆**：结构化事件、情绪、触发点、回应偏好
- **回应强度**：轻回应 / 陪伴 / 复盘 / 建议 / 行动
- **聊后整理**：情绪命名 + 触发点 + 下一步轻动作
- **安全降级**：危机识别 → 现实支持引导

不做范围

不做医疗诊断；不做治疗承诺；不做未成年人主场景；不做角色扮演 / 剧情 / 擦边互动。

里程碑

- 7 月中：Beta MVP 候选版（核心对话 + 场景入口 + 基础记忆 + 聊后整理 + 安全边界）
- 7 月中-8 月中：公开 Beta，验证留存和付费意愿
- 8 月下旬：正式上线，赶在 9 月开学季前

注意：上面的 README 模板里 `\`\`\`` 是为了在本指令文件里能展示出来。实际写入 `README.md` 时去掉所有 `\`` 转义符，恢复成正常的 ````` 三个反引号。

Step 8：根目录 .gitignore + Git 初始化

8.1 创建根目录 `.gitignore`

文件路径：项目根目录 `.gitignore`

```
# 系统
.DS_Store
Thumbs.db

# 编辑器
.vscode/
.idea/
*.swp

# 依赖
node_modules/
.pnpm-store/

# Python
.venv/
__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/
.ruff_cache/

# 环境变量
.env
.env.local

# 构建产物
dist/
build/
.next/
.expo/

# 日志
*.log
logs/
```

8.2 初始化 Git 并首次提交

```
bash
```

```
git init
git add .
git commit -m "chore: initial monorepo scaffold"
```

- apps/mobile: React Native (Expo) shell
- apps/web: Next.js landing page
- services/api: FastAPI backend with health check
- .cursorrules: project context for AI tooling
- README and CLAUDE.md placeholders"

最终验证（必须三个全部通过）

执行以下三件事，确保都成功：

1. 后端能起

```
bash

cd services/api
uv run uvicorn app.main:app --reload
```

新开终端：

```
bash

curl http://localhost:8000/health
# 应返回 {"status":"healthy","env":"dev"}
```

Ctrl+C 停止。

2. 移动端能起

```
bash

cd apps/mobile
pnpm start
```

按 **(i)** 启动 iOS 模拟器，能看到 Expo 默认欢迎页。Ctrl+C 停止。

3. 网页能起

```
bash
```

```
cd apps/web
pnpm dev
```

浏览器访问 <http://localhost:3000> 能看到 Next.js 默认页。Ctrl+C 停止。

完成后的报告格式

完成后请用以下格式向用户报告：

✅ Day 1 搭建完成

[列出实际遇到的问题或调整，如果没有就写"无"]

工具链版本：

- Python: x.x.x
- uv: x.x.x
- Node: x.x.x
- pnpm: x.x.x

已验证可启动：

- ✅ 后端 (<http://localhost:8000>)
- ✅ 移动端 (Expo)
- ✅ 网页 (<http://localhost:3000>)

Git 状态：已完成首次提交, commit hash: xxxxxxxx

下一步：等待用户确认进入 Day 2 (情景记忆 schema 设计 + 人格卡撰写)

重要提醒（请认真遵守）

1. 不要自作主张安装其他包或框架
2. 不要自作主张写业务代码（场景、对话、记忆等）
3. 不要修改既定的目录结构或技术选型
4. 不要接 LLM API（明天的事）
5. 不要做认证系统（明天的事）
6. 不要设计数据库 schema（明天的事，需要单独设计）
7. 遇到任何不在指令里的问题，**停下报告**，不要自行推断

8. 每个 Step 验证不通过就停下，不要硬往下走